

OS LAB EXAM

คู่มือการทำข้อสอบปฏิบัติ

Student Guide

วิชา	Operating Systems
เวลาสอบ	2 ชั่วโมง
คะแนนเต็ม	50 คะแนน
สภาพแวดล้อม	Docker Container (Ubuntu 22.04) บนเครื่องของนักศึกษา
Source Code	https://github.com/prapakorn-lib/t-os

ชื่อ-นามสกุล		รหัสนักศึกษา
กลุ่มเรียน		วันที่สอบ

 ห้ามนำเอกสารออกนอกห้องสอบ | ห้ามใช้ **internet** | ไม่อนุญาตให้ **copy** งานผู้อื่น

0. เตรียมเครื่องก่อนสอบ (ทำก่อนวันสอบ)

⚠️ ส่วนนี้ต้องทำบนเครื่องของนักศึกษาเองก่อนวันสอบ ใช้เวลาประมาณ 10-20 นาที (ขึ้นอยู่กับ internet)

0.1 ติดตั้ง Docker Desktop

ตรวจสอบก่อนว่ามี Docker แล้วหรือยัง:

```
$ docker --version
$ docker compose version
```

✅ ถ้าเห็น **version number** แสดงว่ามีแล้ว ข้ามไปข้อ 0.2 ได้เลย

ถ้ายังไม่มี ให้ดาวน์โหลดและติดตั้งตาม OS:

Windows

```
# 1. ไปที่
https://www.docker.com/products/docker-desktop/
# 2. คลิก Download for Windows
# 3. รัน Docker Desktop
Installer.exe
# 4. ทำตาม Wizard (ใช้ค่า default)
# 5. Restart เครื่องหลังติดตั้งเสร็จ
```

i ต้องการ Windows 10/11 64-bit

macOS

```
# 1. ไปที่
https://www.docker.com/products/docker-desktop/
# 2. เลือก Mac with Apple chip หรือ
Intel chip
# 3. เปิด .dmg และลาก Docker ไป
Applications
# 4. เปิด Docker จาก Applications
# 5. อนุญาต System Extension ถ้าถูกถาม
```

i ต้องการ macOS 12+

Linux

```
#
Ubuntu/Debian:
$ sudo apt update
$ sudo apt install
docker.io
docker-
compose-
plugin -y
$ sudo
usermod -aG
docker $USER
# Log out แล้ว
Log in ใหม่
```

i รัน `newgrp docker` หลัง `usermod`

✓ ทดสอบว่า Docker พร้อมใช้งาน

```
$ docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

✅ ถ้าเห็นข้อความด้านบน แสดงว่า Docker พร้อมแล้ว

0.2 ดาวน์โหลดไฟล์ข้อสอบจาก GitHub

🔗 Repo: <https://github.com/prapakorn-lib/t-os> → โฟลเดอร์ `test_2_2568/docker/`

วิธีที่ 1 — ใช้ `git clone` (แนะนำ)

```
# เปิด Terminal แล้วพิมพ์:
$ git clone https://github.com/prapakorn-lib/t-os.git
$ cd t-os/test_2_2568/docker
$ ls
```

💡 ถ้าไม่มี `git` ให้ติดตั้งก่อน: `Windows` → `https://git-scm.com` | `macOS` → `brew install git` | `Linux` → `sudo apt install git`

วิธีที่ 2 — ดาวน์โหลดเป็น ZIP

1 เปิด **browser** ไปที่ **URL** ด้านล่าง

```
$ https://github.com/prapakorn-lib/t-os
```

2 คลิก **Code** (ปุ่มสีเขียว) → **Download ZIP**

3 แยก **ZIP** แล้วเข้าไปในโฟลเดอร์

```
$ cd t-os-main/test_2_2568/docker
```

ตรวจสอบไฟล์ที่ได้

```
$ ls -la
Dockerfile
docker-compose.yml
broken_script.py
buggy_api.py
entrypoint.sh
main.py
setup_problems.sh
threading_demo.py
```

⚠️ ต้องเห็นไฟล์ครบทั้ง 8 ไฟล์ข้างบน ถ้าไม่ครบให้ดาวน์โหลดใหม่

0.3 Build Docker Image

🕒 ทำครั้งเดียวก่อนวันสอบ ใช้เวลา 5-10 นาที ต้องการ *internet*

ให้อยู่ในโฟลเดอร์ `test_2_2568/docker/` ก่อนแล้วรันคำสั่งนี้:

```
$ docker build -t os-lab-exam .
```

ระหว่าง **build** จะเห็นข้อความวิ่งขึ้นมาเยอะ รอจนเห็นบรรทัดสุดท้าย:

```
Successfully built xxxxxxxx
Successfully tagged os-lab-exam:latest
```

ตรวจสอบว่า **build** สำเร็จ:

```
$ docker images | grep os-lab-exam
os-lab-exam  latest  xxxxxxxx  ...  about 1GB
```

✅ เห็น **os-lab-exam** ในรายการ = พร้อมสอบแล้ว ✅

1. วันสอบ — เริ่มต้นใช้งาน Docker

1.1 Start Container

🔴 เปิด **Terminal** (หรือ **PowerShell** บน **Windows**) แล้วเข้าโฟลเดอร์ `test_2_2568/docker/` ก่อน

วิธีที่ 1 — ใช้ **docker-compose** (แนะนำ)

```
$ cd t-os/test_2_2568/docker
$ docker compose up -d
$ docker exec -it os-lab-exam bash
```

วิธีที่ 2 — ใช้ **docker run** โดยตรง

```
$ docker run -it --name os-lab-exam -p 8000:8000 -p 8001:8001 os-lab-exam
```

1.2 สิ่งที่จะเห็นเมื่อ **Container Start** สำเร็จ

```
=====
OS Lab Exam Environment
Ubuntu 22.04 | Docker Container
=====
[init] Starting PostgreSQL...
[init] Setting up exam problems...
[init] Environment ready!

ไฟล์สำหรับการสอบอยู่ที่: /exam/
ls /exam/ เพื่อดูรายการไฟล์

root@lab-server:/home/student#
```

✅ เห็น `root@lab-server:/home/student#` แสดงว่าพร้อมทำข้อสอบแล้ว

1.3 การเปิด **Terminal** เพิ่มเติม (สำคัญ!)

ระหว่างสอบอาจต้องการ **terminal 2** อันพร้อมกัน เช่น อัน 1 รัน **server** อัน 2 ทดสอบ

⚠️ อย่า **Ctrl+C** ปิด **terminal** แรก ให้เปิด **terminal** ใหม่แล้วรันคำสั่งด้านล่าง

```
# เปิด Terminal ใหม่ แล้วพิมพ์:
$ docker exec -it os-lab-exam bash
```

1.4 คำสั่งที่ใช้บ่อยใน **Container**

คำสั่ง	ใช้ทำอะไร
<code>ls /exam/</code>	ดูไฟล์ข้อสอบทั้งหมด
<code>nano /path/file.py</code>	แก้ไขไฟล์ (บันทึก: Ctrl+X → Y → Enter)
<code>python3 file.py</code>	รัน Python script
<code>ls -la /path/</code>	ดูไฟล์พร้อม permission
<code>mkdir -p /path/dir</code>	สร้าง directory (รวม parent)
<code>chown user:group /path</code>	เปลี่ยนเจ้าของไฟล์

chmod 750 /path	ตั้งค่า permission
service postgresql status	ตรวจสอบสถานะ PostgreSQL
lsof -i :8000	ดู process ที่ใช้ port 8000
curl http://localhost:8000/	ทดสอบ API จาก terminal
exit หรือ Ctrl+D	ออกจาก container (container ยังรันอยู่)

1.5 การแบ่งเวลาแนะนำ

ส่วน	หัวข้อ	คะแนน	เวลา
1	การติดตั้งและตั้งค่าระบบ (3 ข้อ)	15	30 นาที
2	การเขียนโปรแกรม (3 ข้อ)	15	40 นาที
3	Web API Service (3 ข้อ)	20	50 นาที
	รวม	50	120 นาที

2. ส่วนที่ 1 — การติดตั้งและตั้งค่าระบบ (15 คะแนน)

ข้อ 1.1 — ตั้งค่า User และสิทธิ์ (5 คะแนน)

✦ *container* มี user 'student' อยู่แล้ว แต่ยังไม่มี *devuser* และยังไม่มี */opt/myapp*

สิ่งที่ต้องทำ

1. สร้าง user ใหม่ชื่อ *devuser* พร้อมตั้ง password
2. เพิ่ม *devuser* เข้ากลุ่ม *sudo*
3. สร้างไดเรกทอรี */opt/myapp* และกำหนดให้ *devuser* เป็นเจ้าของ
4. ตั้งค่า permission ของ */opt/myapp* เป็น 750
5. ตรวจสอบผลด้วย `ls -la /opt/`

Hint — คำสั่งที่เกี่ยวข้อง

```
$ useradd -m -s /bin/bash [username]
$ echo '[user]:[password]' | chpasswd
$ usermod -aG sudo [username]
$ mkdir -p /path
$ chown [user]:[group] /path
$ chmod 750 /path
```

📄 บันทึกคำสั่งที่ใช้และผลที่ได้:

เกณฑ์การให้คะแนน	คะแนนที่ได้
สร้าง <i>devuser</i> สำเร็จ	____ / 1
เพิ่มเข้ากลุ่ม <i>sudo</i> สำเร็จ	____ / 1
สร้าง <i>/opt/myapp</i> และ owner ถูกต้อง	____ / 2
permission 750 ถูกต้อง	____ / 1

ข้อ 1.2 — ติดตั้ง Development Tools (5 คะแนน)

✦ *container* มี *python3* ติดตั้งแล้ว แต่ *build-essential* และ *git* ยังไม่มี

สิ่งที่ต้องทำ

6. อัปเดต package list ด้วย `apt update`
7. ติดตั้ง *build-essential* และ *git*
8. ตรวจสอบ version ของ *python3*, *pip3*, *gcc*, *git*
9. เพิ่ม environment variable `APP_ENV=production` ใน `~/.bashrc`
10. ตรวจสอบว่า *APP_ENV* ถูกตั้งค่าแล้วด้วย `echo $APP_ENV`

Hint — คำสั่งที่เกี่ยวข้อง

```
$ apt update
```

```
$ apt install -y build-essential git
$ echo 'export APP_ENV=production' >> ~/.bashrc
$ source ~/.bashrc
$ echo $APP_ENV
```

 บันทึกผลการตรวจสอบ **version** ของทุก **tool**:

เกณฑ์การให้คะแนน	คะแนนที่ได้
ติดตั้ง build-essential และ git สำเร็จ	____ / 2
แสดง version ทั้ง 4 tools ได้	____ / 2
ตั้งค่า APP_ENV=production ถาวรสำเร็จ	____ / 1

ข้อ 1.3 — แก้ปัญหา: Service ไม่ทำงาน (5 คะแนน)

 **container** มีปัญหา 2 จุดที่ซ่อนไว้ตั้งแต่ต้น นักศึกษาต้องวินิจฉัยและแก้ไขเอง

ปัญหาที่ 1 — PostgreSQL ไม่สามารถ start ได้

ตรวจสอบด้วย `service postgresql status` แล้วหาสาเหตุ แก้ไข และยืนยันว่า `start` สำเร็จ

 **Hint:** ปัญหาเกิดจาก *permission* หรือ *ownership* ของ *data directory* ของ *PostgreSQL*

ปัญหาที่ 2 — Port 8000 ถูกใช้งานอยู่

ตรวจสอบว่า `process` ใดกำลัง `bind port 8000` อยู่ หยุด `process` นั้น และยืนยันว่า `port` ว่างแล้ว

 **Hint:** ลองใช้ `lsof -i :8000` หรือ `ss -tlnp | grep 8000` เพื่อหา `process`

 บันทึกวิธีวินิจฉัยและแก้ไขปัญหทั้ง 2 จุด:

เกณฑ์การให้คะแนน	คะแนนที่ได้
วินิจฉัยปัญหา PostgreSQL และระบุสาเหตุได้	____ / 1
แก้ไขและ start PostgreSQL สำเร็จ	____ / 1.5
วินิจฉัยและหา process ที่ใช้ port 8000 ได้	____ / 1
kill process และยืนยัน port ว่างแล้ว	____ / 1.5

3. ส่วนที่ 2 — การเขียนโปรแกรม (15 คะแนน)

ข้อ 2.1 — File Management Script (5 คะแนน)

✦ เขียน *Python script* เองใน `/opt/myapp/` (ต้องสร้าง *directory* นี้ในข้อ 1.1 ก่อน)

สิ่งที่ต้องทำ

เขียน `file_manager.py` ใน `/opt/myapp/` ที่ทำสิ่งต่อไปนี้:

11. สร้างไฟล์ `app.log` พร้อม `timestamp` ปัจจุบัน
12. เขียนข้อความ 'System started' ลงในไฟล์
13. อ่านและแสดงเนื้อหาของไฟล์
14. แสดง `file size` และ `permission` ด้วย `os.stat()`

Hint — Module ที่เกี่ยวข้อง

```
$ import os, datetime
$ datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
$ open('/path/file', 'w')      # เปิดเขียน
$ open('/path/file', 'r')      # เปิดอ่าน
$ os.stat('/path/file').st_size      # ขนาดไฟล์
$ oct(os.stat('/path/file').st_mode) # permission
```

📄 บันทึก **output** ที่ได้เมื่อรัน `python3 /opt/myapp/file_manager.py`:

เกณฑ์การให้คะแนน	คะแนนที่ได้
script รันได้โดยไม่มี error	____ / 1
สร้างและเขียน <code>app.log</code> สำเร็จ พร้อม <code>timestamp</code>	____ / 2
อ่านไฟล์และแสดง <code>size</code> + <code>permission</code> ได้	____ / 2

ข้อ 2.2 — ทดสอบ Multi-threading (5 คะแนน)

✦ ใช้ *script* ที่อยู่ใน `/exam/threading_demo.py` ไม่ต้องเขียน *code* เอง

สิ่งที่ต้องทำ

1	รัน <code>threading_demo.py</code> และดู output
	<pre>\$ python3 /exam/threading_demo.py</pre>
2	รันซ้ำ 3 ครั้งและสังเกตความแตกต่าง
	<pre>\$ for i in 1 2 3; do echo '== Run '\$i' =='; python3 /exam/threading_demo.py; done</pre>

3 เขียนคำตอบในไฟล์ **threading_answer.txt**

```
$ nano /opt/myapp/threading_answer.txt
```

คำถามที่ต้องตอบใน **threading_answer.txt**

- script นี้สร้าง thread กี่ตัว?
- แต่ละ thread ทำอะไร?
- ลำดับ output ที่ได้แต่ละครั้งเหมือนกันหรือไม่ เพราะอะไร?

 สรุปสิ่งที่สังเกตได้ (2-3 ประโยค):

เกณฑ์การให้คะแนน	คะแนนที่ได้
รัน script และบันทึก output สำเร็จ	____ / 1
ระบุจำนวน thread และหน้าที่ของแต่ละ thread ได้	____ / 2
อธิบายเหตุผลที่ลำดับ output ไม่เหมือนกันแต่ละครั้งได้	____ / 2

ข้อ 2.3 — แก้ปัญหา: Script Error (5 คะแนน)

 ใช้ `/exam/broken_script.py` มี bug อยู่ 1 จุด อ่าน **error** แล้วแก้ไข

สิ่งที่ต้องทำ

1 รัน script เพื่อดู error

```
$ python3 /exam/broken_script.py
```

2 Copy script มาแก้ไข

```
$ cp /exam/broken_script.py /opt/myapp/broken_script.py
$ nano /opt/myapp/broken_script.py
```

3 รันจนได้ output ที่ถูกต้อง

```
$ python3 /opt/myapp/broken_script.py
```



Hint: อ่าน **error message** ให้ละเอียด มันจะบอกชัดเจนว่าปัญหาอยู่ที่ไหน

 บันทึก **error** ที่พบ / สาเหตุ / สิ่งแก้ไข / output หลังแก้ไข

เกณฑ์การให้คะแนน	คะแนนที่ได้
อ่าน error message และระบุสาเหตุได้	____ / 2
แก้ไข bug สำเร็จ script รันได้โดยไม่ error	____ / 2
อธิบายสิ่งที่แก้ไขและเหตุผลได้	____ / 1

4. ส่วนที่ 3 — Web API Service (20 คะแนน)

ข้อ 3.1 — ติดตั้งและตั้งค่า PostgreSQL (5 คะแนน)

🔴 PostgreSQL ติดตั้งในระบบแล้ว แต่ยังไม่มีการสร้าง *database* และ *user* สำหรับงาน

สิ่งที่ต้องทำ

15. ตรวจสอบและ start PostgreSQL
16. สร้าง database ชื่อ labdb และ user ชื่อ labuser พร้อมตั้ง password
17. Grant สิทธิ์ทั้งหมดให้ labuser บน labdb
18. สร้าง table products ที่มี columns: id, name, price, stock
19. Insert ข้อมูลตัวอย่าง 3 รายการ

Hint — SQL Commands

```
$ sudo -u postgres psql          # เข้า psql
$ CREATE USER labuser WITH PASSWORD 'xxx';
$ CREATE DATABASE labdb OWNER labuser;
$ GRANT ALL PRIVILEGES ON DATABASE labdb TO labuser;
$ \q                             # ออกจาก psql
$ psql -U labuser -d labdb -h localhost # เชื่อมต่อเป็น labuser
```

📄 บันทึก SQL ที่ใช้และผล `SELECT * FROM products;`

เกณฑ์การให้คะแนน	คะแนนที่ได้
สร้าง database และ user สำเร็จ	____ / 1
Grant สิทธิ์ถูกต้อง	____ / 1
สร้าง table products ด้วย columns ครบถ้วน	____ / 2
Insert ข้อมูล 3 รายการสำเร็จ	____ / 1

ข้อ 3.2 — ตั้งค่าและรัน FastAPI Application (10 คะแนน)

🔴 มี *main.py* เตรียม code ไว้ที่ `/exam/main.py` แต่ยังไม่ได้ตั้งค่า *database*

สิ่งที่ต้องทำ

1 Copy main.py ไปที่ /opt/myapp/

```
$ cp /exam/main.py /opt/myapp/main.py
```

2 แก้ไข DB_CONFIG ให้ตรงกับข้อ 3.1

```
$ nano /opt/myapp/main.py
# แก้บรรทัด :
# DB_NAME      = '???'   ← ใส่ชื่อ database
# DB_USER      = '???'   ← ใส่ชื่อ user
# DB_PASSWORD  = '???'   ← ใส่ password
```

3 รัน FastAPI server

```
$ cd /opt/myapp
$ uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

4 เปิด Terminal ใหม่ แล้วทดสอบ 4 endpoints

```
$ curl http://localhost:8000/
$ curl http://localhost:8000/products
$ curl http://localhost:8000/products/1
$ curl -X POST http://localhost:8000/products -H 'Content-Type: application/json' -d '{"name": "Monitor", "price": 8000, "stock": 5}'
```

💡 เปิด terminal เพิ่ม: `docker exec -it os-lab-exam bash`

📄 บันทึก output ของแต่ละ curl command:

Endpoint	Response / Status	คะแนน
GET /		____ / 2
GET /products		____ / 3
GET /products/{id}		____ / 3
POST /products		____ / 2

ข้อ 3.3 — แก้ปัญหา: API ไม่ตอบสนอง 🛠️ (5 คะแนน)

🛠️ มี `buggy_api.py` ที่ `/exam/` มี bug 2 จุด ต้องแก้ไขให้ `GET /items` ตอบกลับได้

สิ่งที่ต้องทำ

1 Copy buggy_api.py มาแก้ไข

```
$ cp /exam/buggy_api.py /opt/myapp/buggy_api.py
```

2 รันและดู error

```
$ cd /opt/myapp
$ uvicorn buggy_api:app --port 8001
```

3 ทดสอบเพื่อดู **error message**

```
$ curl http://localhost:8001/items
```

4 แก้ไข **bug** ที่ละจุดจนกว่า **API** จะ **response** ถูกต้อง

```
$ nano /opt/myapp/buggy_api.py
```

 **Hint:** มี **bug 2** จุด — ลองดูที่ **database config** และ **SQL query**

 บันทึก **bug** ทั้ง **2** จุด / วิธีแก้ / **output** หลังแก้:

เกณฑ์การให้คะแนน	คะแนนที่ได้
แก้ไข bug จุดที่ 1 และระบุสาเหตุได้	____ / 2
แก้ไข bug จุดที่ 2 และระบุสาเหตุได้	____ / 2
GET /items response ถูกต้อง ได้ products list	____ / 1

5. สรุปคะแนนและ Troubleshooting

5.1 ตารางสรุปคะแนน

ข้อ	หัวข้อ	คะแนนเต็ม	คะแนนที่ได้
1.1	ตั้งค่า User และสิทธิ์	5	
1.2	ติดตั้ง Development Tools	5	
1.3	แก้ปัญหา Service ไม่ทำงาน	5	
2.1	File Management Script	5	
2.2	ทดสอบ Multi-threading	5	
2.3	แก้ปัญหา Script Error	5	
3.1	ติดตั้งและตั้งค่า PostgreSQL	5	
3.2	ตั้งค่าและรัน FastAPI Application	10	
3.3	แก้ปัญหา API ไม่ตอบสนอง	5	
รวม		50	

5.2 Troubleshooting — ปัญหาที่พบบ่อย

ปัญหา	วิธีแก้
docker: command not found	ติดตั้ง Docker Desktop ใหม่ หรือ restart เครื่องแล้วลองใหม่
Cannot connect to Docker daemon	เปิด Docker Desktop application ก่อน แล้วรอ icon ใน taskbar
Port 8000 already in use (บนเครื่องตัวเอง)	เปลี่ยน port: docker run ... -p 8080:8000 ... แล้วใช้ curl localhost:8080
image not found: os-lab-exam	รัน docker build -t os-lab-exam . อีกครั้ง ตรวจสอบว่าอยู่ในโฟลเดอร์ที่มี Dockerfile
container ค้าง / ไม่ตอบสนอง	เปิด Terminal ใหม่แล้วรัน: docker restart os-lab-exam && docker exec -it os-lab-exam bash
ต้องการ reset container ใหม่ทั้งหมด	docker rm -f os-lab-exam && docker run -it --name os-lab-exam -p 8000:8000 os-lab-exam

✓ ตรวจสอบให้แน่ใจว่าบันทึกผลทุกข้อแล้วก่อนแจ้งอาจารย์ว่าทำเสร็จ

ลายมือชื่อนักศึกษา

ลายมือชื่อผู้คุมสอบ / ผู้ตรวจ

()

()